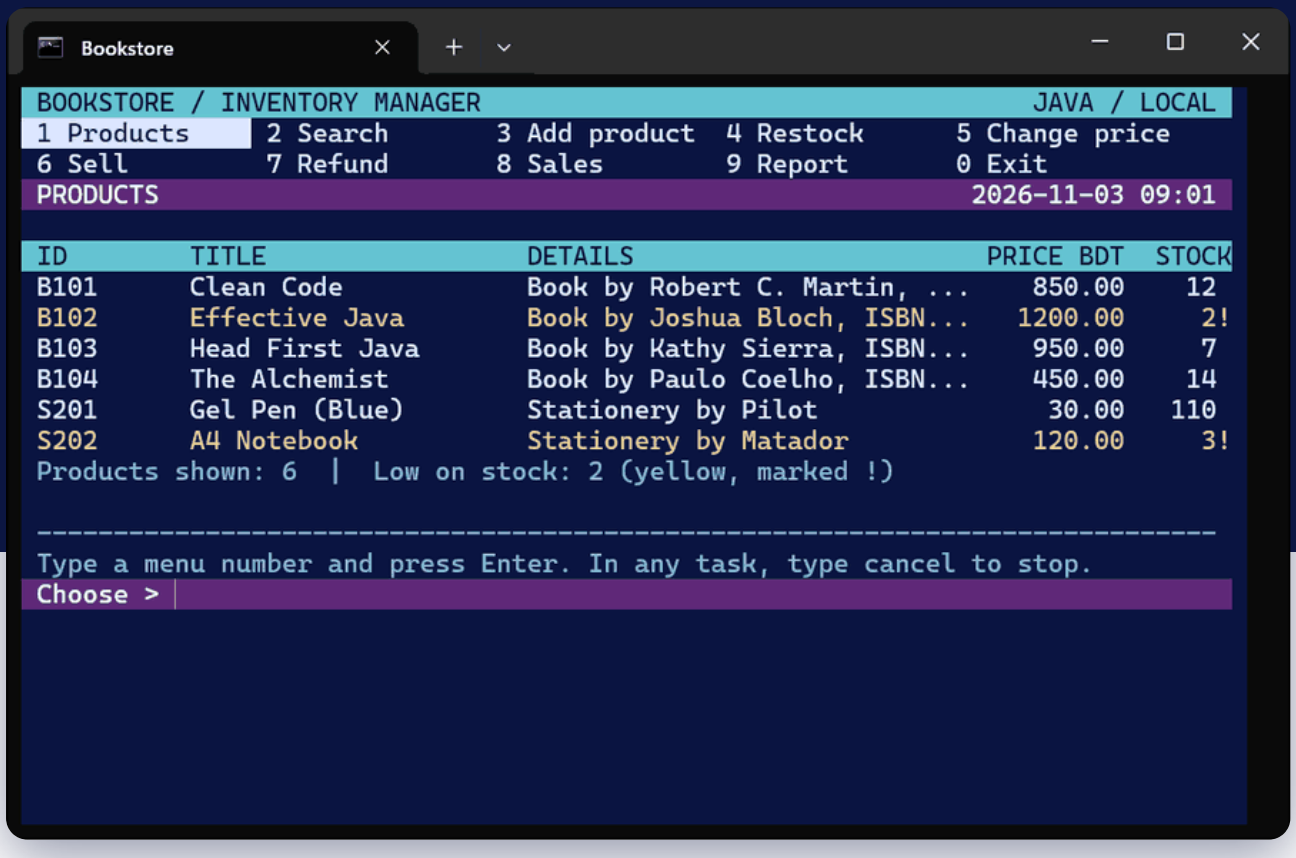
OBJECT-ORIENTED PROGRAMMING · PROJECT REPORT

Bookstore Inventory Manager

A Java console application for a small bookstore



LANGUAGE
Java 17

CODE
4 files · 635 lines

DATE
November 2026

Source code: https://github.com/AtlasRoX/Bookstore-JAVA

Contents

1 Introduction 2

2 Features 2

3 Tools and Technologies 3

4 Project Structure and How to Run 3

5 How It Works 3

6 Screens 4

7 OOP Concepts Used 5

- 7.1 Abstraction 6
- 7.2 Encapsulation 6
- 7.3 Inheritance 7
- 7.4 Polymorphism 7
- 7.5 Static vs non-static 8
- 7.6 Method overloading 8
- 7.7 Method overriding 8
- 7.8 The this keyword 9
- 7.9 The super keyword 9
- 7.10 Subclass vs superclass reference 9
- 7.11 Diamond problem resolution 10

8 Key Code Behind the Screens 11

9 Limitations and Future Work 12

10 Conclusion 12

1 Introduction

Bookstore Inventory Manager is a Java 17 console application for a small bookstore that sells books and stationery. Staff can add, search and restock products, change prices, sell with an optional 5% member discount, refund sales, and see the sales history and a business report. Every change is saved automatically to `bookstore.dat`.

Why this project

Many small bookstores still track their stock and sales in a notebook or a spreadsheet. Counting stock by hand, calculating discounts and change, and finding an old sale for a refund are slow and easy to get wrong. This program does that work: it always knows the stock, calculates totals and change, keeps every receipt, and shows the income at a glance.

A bookstore is also a natural fit for Object-Oriented Programming. Books and stationery share the same basic data (ID, title, price, stock) but differ in their details (author and ISBN, or brand). That leads directly to an abstract superclass with two subclasses, and from there to inheritance, overriding, polymorphism and the other concepts of the course, each one used by a real feature rather than added only for show.

Objectives

- Manage two product types (books and stationery) that share common data but differ in their details.
- Handle the daily work of a shop: adding products, restocking, changing prices, selling, refunding and reporting.
- Check every answer, so the program never crashes and wrong data is never saved.
- Keep the code short and easy to read: four files, each with one clear job.
- Use all 11 OOP concepts of the course, each inside a real feature.

2 Features

All work is done from one menu. The user types a key and presses Enter; the menu tabs at the top of every screen show the keys.

Key	Feature	What it does
1	**Products**	Lists every product with ID, title, details, price and stock; low stock (3 or fewer) is highlighted in yellow with “!”.
2	**Search**	Finds products by any part of the ID, title, author, brand or ISBN, ignoring upper and lower case.
3	**Add product**	Adds a book (with an ISBN, or “-” if it has none) or a stationery item; an ID that already exists is refused.
4	**Restock**	Adds received units to a product’s stock.
5	**Change price**	Sets a new unit price; old receipts keep the price that was paid.
6	**Sell**	Sells one product: checks the quantity against the stock, applies the 5% member discount, checks the cash, shows the change and prints a receipt.
7	**Refund**	Refunds a whole receipt once; the units go back into stock.
8	**Sales**	Shows every receipt with date, product, quantity, total, change and status; refunded receipts are yellow.
9	**Report**	Shows income after refunds, units sold, number of products, low-stock count, units in stock, products added this session and stock value.
0	**Exit**	Closes the program; every change was already saved when it was made.

Every question checks its answer: a wrong answer (an unknown ID, too little cash, a letter instead of a number) is explained in a red bar and asked again, and typing `cancel` stops the current task without changing anything.

3 Tools and Technologies

Tool / technology	Used for
Java 17 (JDK: `javac`, `java`)	Writing, compiling and running the program
Java standard library	`ArrayList`, `Scanner`, `String.format`, `LocalDate` / `LocalDateTime`, object streams
Java object serialization	Saving and loading all products and sales in one step
ANSI escape codes	Colors and clearing the screen in the terminal
Windows Terminal + `run.bat`	Opening the program in a window sized exactly 80 × 24 characters
Git and GitHub	Version control; source code at github.com/AtlasRoX/Bookstore-JAVA

4 Project Structure and How to Run

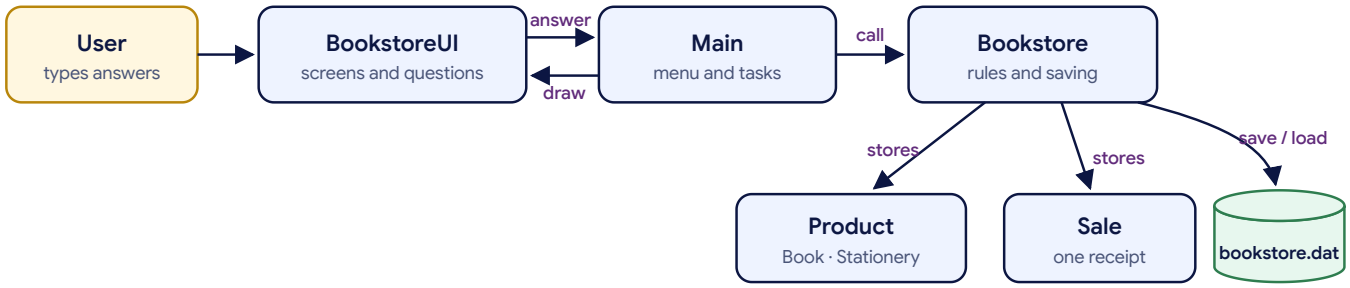
File	Lines	Job
`Product.java`	126	Interfaces Item, Trackable, Promotable; abstract class Product; subclasses Book and Stationery
`Bookstore.java`	139	Shop rules (add, restock, sell, refund, search), saving and loading; class Sale (one receipt)
`BookstoreUI.java`	222	Everything on screen: colors, menu tabs, tables, receipt, questions
`Main.java`	148	Starts the program, runs the menu and one method per task

The classes work in layers: `BookstoreUI` shows a screen and reads the answer → `Main` decides which task runs → `Bookstore` checks the rules, changes the data and saves it → `Product` and `Sale` hold the data → `BookstoreUI` shows the result.

> **Run:** double-click `run.bat`, which compiles the program and opens it in an 80 × 24 window; or, in the project folder, `javac *.java` then `java Main`. Requires JDK 17 or newer.

5 How It Works

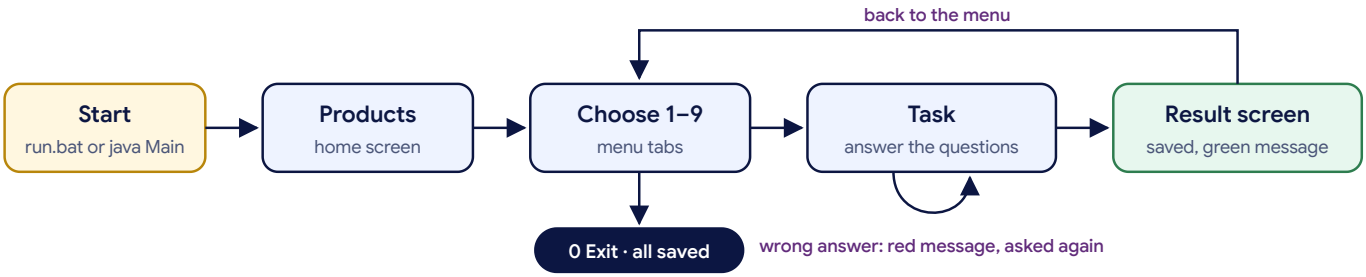
The program is split into layers, so each class has one job. The user only talks to `BookstoreUI`; `Main` decides what happens; `Bookstore` checks the shop's rules and saves; `Product` and `Sale` are the data.



How the parts of the program work together.

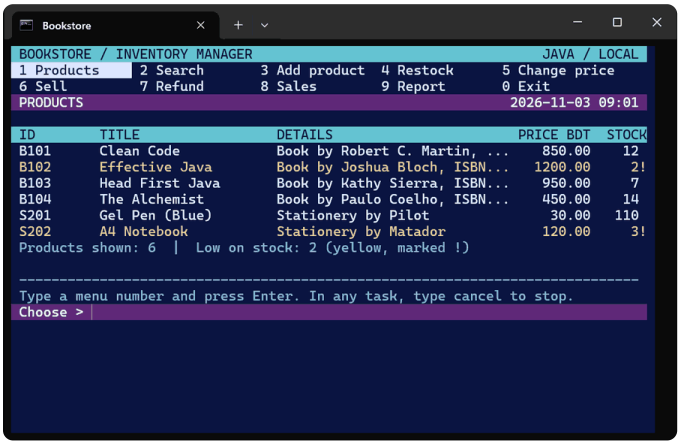
1. **Start.** `run.bat` compiles the code and opens the window; `main()` loads `bookstore.dat` (or starts an empty shop the first time).
2. **Menu.** The Products screen appears and the menu loop asks "Choose".
3. **Task.** A `switch` in `Main` runs the chosen task; there is one method per menu key.
4. **Questions.** The task asks its questions through `BookstoreUI`; a wrong answer is asked again, and `cancel` stops the task.

5. **Rules and saving.** `Bookstore` checks its rules, changes the data and saves the whole shop to `bookstore.dat` at once.
6. **Result.** The screen is redrawn with the result in green, or a broken rule is shown in red; then the program returns to the menu.
7. **Exit.** Key 0 closes the program. Nothing is lost, because every change was saved the moment it was made.

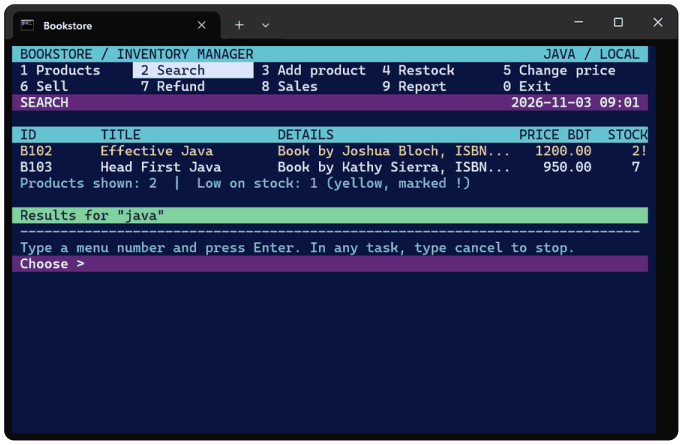


User flow: from start to the result screen and back to the menu.

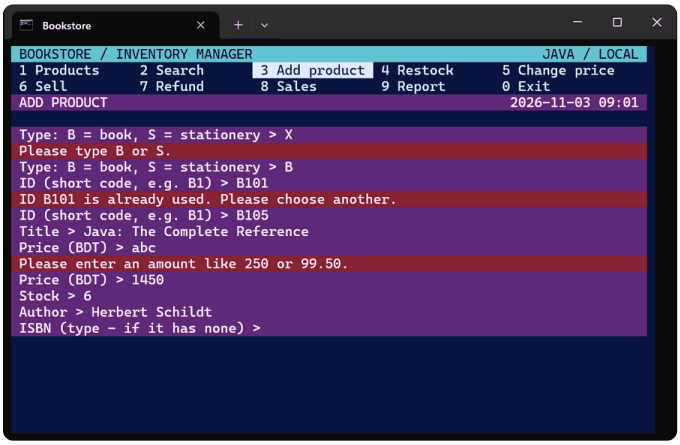
6 Screens



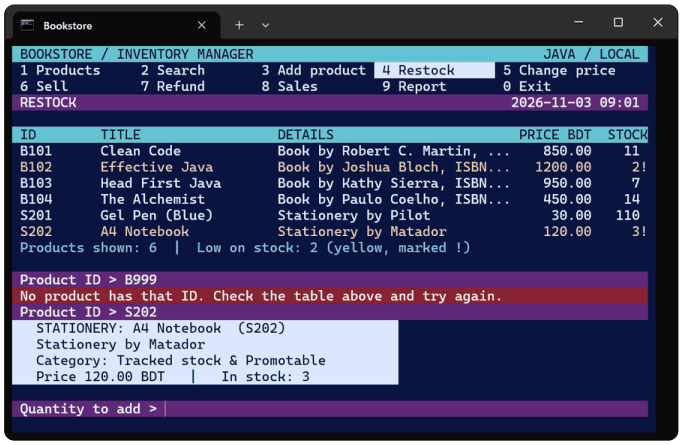
1 Products — the home screen: every product with its details, price and stock. Low stock (3 or fewer) is yellow with "!".



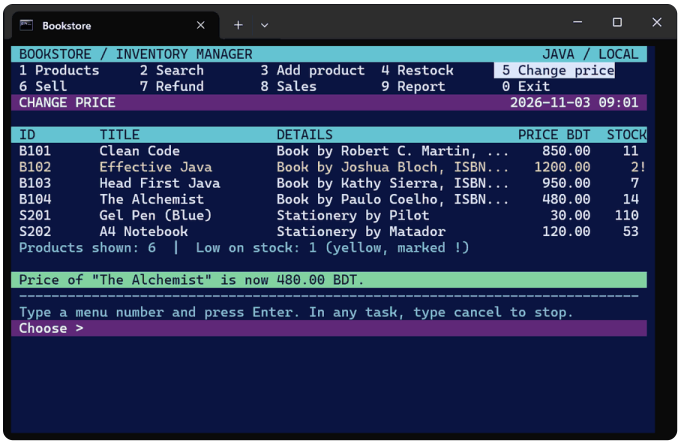
2 Search — finds products by any part of the ID, title, author, brand or ISBN.



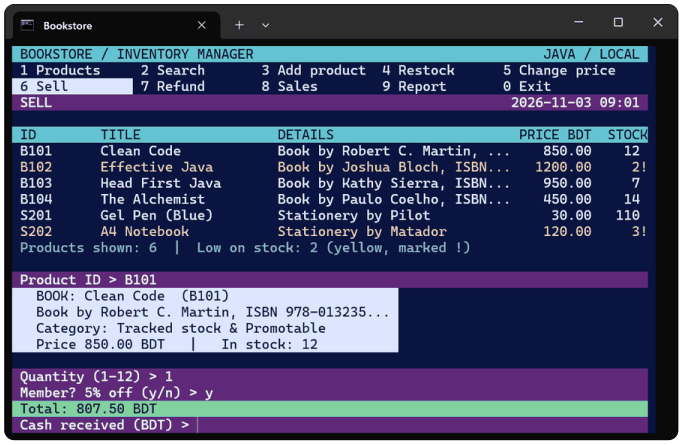
3 Add product — one question at a time; each wrong answer is explained in red and asked again.



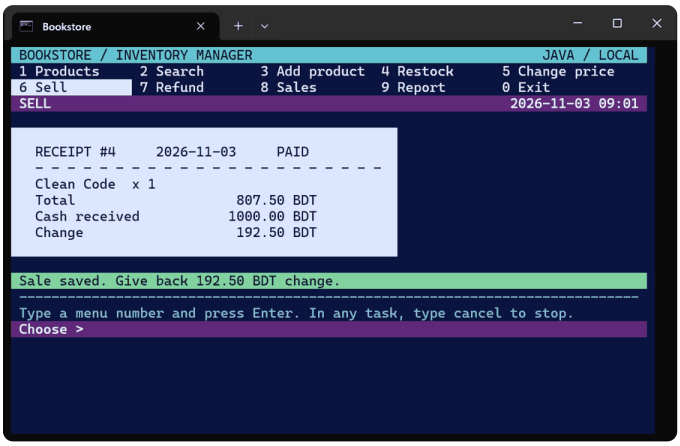
4 Restock — an unknown ID is refused; the card shows type, details, category and current stock.



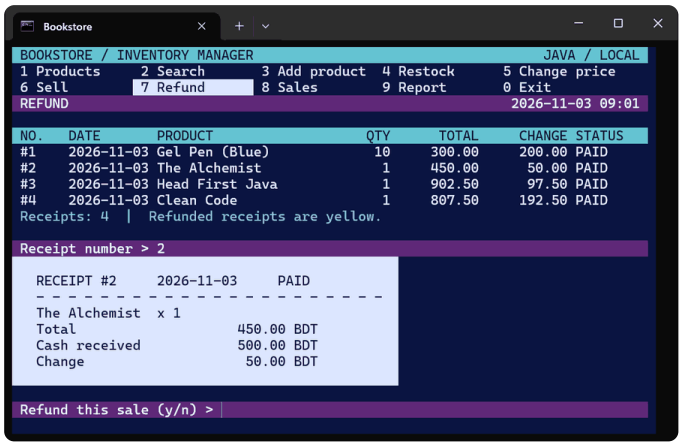
5 Change price — the new price is saved and shown in the table with a green message.



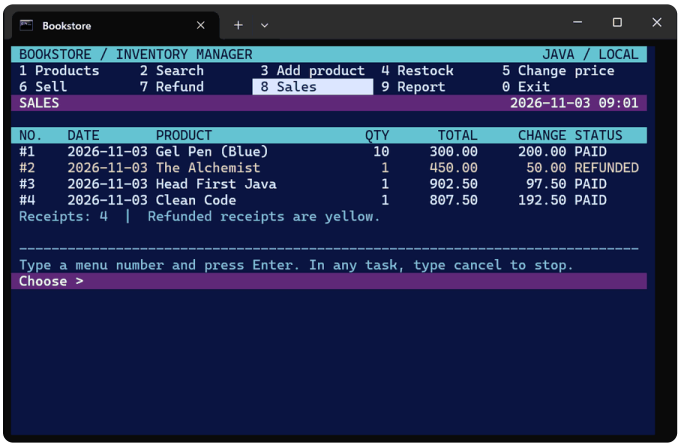
6 Sell — the quantity range is shown; a member gets 5% off; the total appears in green.



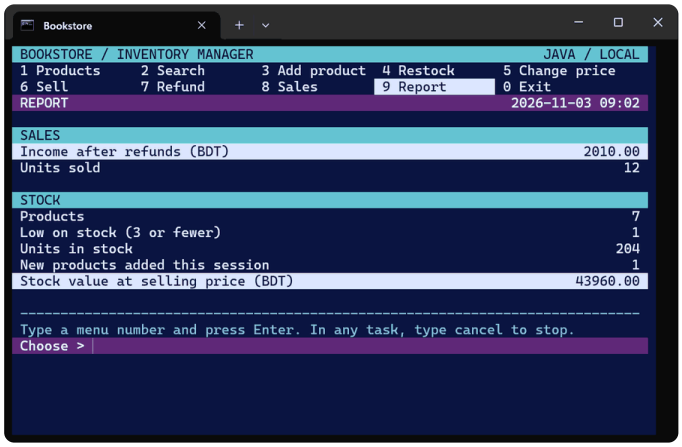
6 Sell — after payment the screen shows the receipt and the change to give back.



7 Refund — choose a receipt and confirm; the units go back into stock.



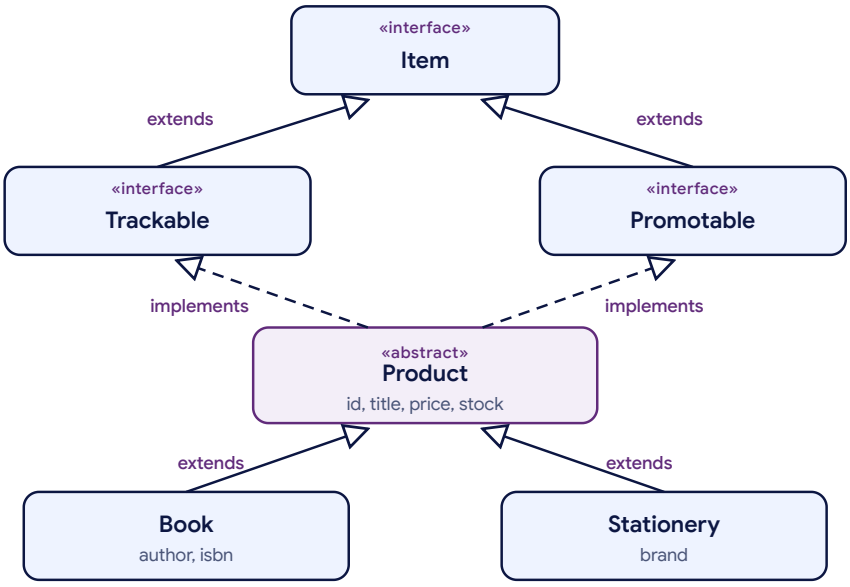
8 Sales — every receipt with total, change and status; refunded ones are yellow.



9 Report — income after refunds, units sold, stock, products added this session, stock value.

7 OOP Concepts Used

The product classes form the main hierarchy. Hollow arrows point to the parent: solid lines mean *extends*, dashed lines mean *implements*. Each subclass adds only its own fields.



Class hierarchy of the products.

7.1 Abstraction

Abstraction means showing only what is needed and hiding the details. `Product` is declared `abstract`, so `new Product(...)` does not compile: the shop can only create a `Book` or a `Stationery`. `getType()` and `getDetails()` have no body here, so every product must be able to describe itself, but each type decides how. The tables, search and receipts work only with `Product` and never need to know the exact type.

Product.java lines 18–77

```
18 // ABSTRACTION: Product is abstract, so the program can only make a Book or a Stationery.
19 // ENCAPSULATION: the fields are private and are used only through getters and setters.
20 public abstract class Product implements Trackable, Promotable, Serializable {
…
75 // ABSTRACT METHODS: every subclass must write its own version (overriding).
76 public abstract String getType();
77 public abstract String getDetails();
```

7.2 Encapsulation

Encapsulation means keeping an object's data private and changing it only through its own methods. The fields of `Product` are `private`; other classes read them with getters and change them with setters, which refuse a negative price or stock, so a product can never hold impossible values. `Bookstore` also hides its lists of products and sales, so only its own methods (add, sell, refund …) can change them, and every change is saved.

Product.java lines 32–62

```
32 // NON-STATIC (instance) variables: every product object has its own copy.
33 private String id, title;
34 private double price;
35 private int stock;
…
49 // NON-STATIC METHODS: getters and setters work on one object's own data.
50 public String getId() { return id; }
51 public String getTitle() { return title; }
52 public double getPrice() { return price; }
53 public int getStock() { return stock; }
55 // The setters protect the data: a negative value is refused.
56 public void setPrice(double price) {
57     if (price >= 0) this.price = price;
58 }
60 public void setStock(int stock) {
61     if (stock >= 0) this.stock = stock;
62 }
```

Bookstore.java lines 8–14

```java
private static final String FILE = "bookstore.dat"; // where everything is saved
private ArrayList<Product> products = new ArrayList<>(); // all products
private ArrayList<Sale> sales = new ArrayList<>(); // all receipts
// Getters for the two lists.
public ArrayList<Product> getProducts() { return products; }
public ArrayList<Sale> getSales() { return sales; }
```

7.3 Inheritance

Inheritance lets a subclass reuse everything from its superclass. `Book` and `Stationery` `extend` `Product`, so they get id, title, price, stock, all getters and setters and `calculateDiscount()` without writing them again. Each subclass only adds what is special about it: a book has an author and an ISBN, a stationery item has a brand. Because both are a `Product`, the inventory and every screen treat them the same way.

Product.java lines 80–118

```java
// INHERITANCE: Book is a subclass of the superclass Product.
class Book extends Product {
    private static final long serialVersionUID = 1L;
    private String author, isbn;

public String getAuthor() { return author; } // only a Book has an author

// INHERITANCE: Stationery is another subclass of Product.
class Stationery extends Product {
    private static final long serialVersionUID = 1L;
    private String brand;

public String getBrand() { return brand; } // only Stationery has a brand
```

7.4 Polymorphism

Polymorphism means one call, many forms. In `showProducts` every item is held in a `Product` variable, yet `p.getDetails()` runs Book's version ("Book by …, ISBN …") for books and Stationery's version ("Stationery by …") for stationery: Java chooses the method of the real object while the program runs (runtime polymorphism). The product card does the same with `p.getType()`. Method overloading (7.6) is the compile-time form of polymorphism.

BookstoreUI.java lines 134–166

```java
// A table of products. Low stock (3 or fewer) is yellow and marked with "!".
public void showProducts(ArrayList<Product> list) {
    line(CYAN, String.format(" %-9s %-21s %-29s %9s %6s", "ID", "TITLE", "DETAILS", "PRICE BDT",
"STOCK"));
    int low = 0;
    for (Product p : list) {
        String color = BODY;
        String stock = p.getStock() + " ";
        if (p.getStock() <= 3) {
            color = YELLOW;
            stock = p.getStock() + "!";
            low++;
        }
        line(color, String.format(" %-9s %-21s %-29s %9.2f %6s",
                fit(p.getId(), 9), fit(p.getTitle(), 21), fit(p.getDetails(), 29), p.getPrice(),
stock));
    }
    if (list.isEmpty()) line(DIM, " No products found. Choose 3 to add one.");
    line(DIM, " Products shown: " + list.size() + "  |  Low on stock: " + low + " (yellow, marked !)");
    line(BODY, "");
}

// The white product card: type, details, category, price and stock.
public void showProduct(Product p) {
    card(p.getType() + ": " + p.getTitle() + "  (" + p.getId() + ")");
    card(p.getDetails());
    card("Category: " + p.getCategory());
    card(String.format("Price %.2f %s   |   In stock: %d", p.getPrice(), Product.CURRENCY,
p.getStock()));
    line(BODY, "");
}
```

7.5 Static vs non-static

A `static` member belongs to the class and is shared by all objects; a non-static member belongs to each object. `CURRENCY` is one shared text, set once in a `static` block when Java loads the class. `createdCount` is a shared counter that every constructor call increases; the static method `Product.getCreatedCount()` reads it and the Report shows it as "New products added this session". `price` and `stock` are non-static: every product has its own. `Main` uses only static members, so no `Main` object is ever created.

Product.java lines 23–47

```
23  // STATIC variables belong to the class and are shared by every product.
24  public static final String CURRENCY;
25  private static int createdCount = 0; // products created while the program runs (shown in the Report)
27  // STATIC BLOCK: runs once, when Java first loads the Product class.
28  static {
29      CURRENCY = "BDT";
30  }
…
43  createdCount++;
…
46  // STATIC METHOD: called on the class itself, e.g. Product.getCreatedCount()
47  public static int getCreatedCount() { return createdCount; }
```

BookstoreUI.java lines 164–218

```
164 card(String.format("Price %.2f %s   |   In stock: %d", p.getPrice(), Product.CURRENCY, p.getStock()));
…
218 line(BODY, String.format(" %-58s%19d", "New products added this session", Product.getCreatedCount()));
```

Main.java lines 4–6

```
4   public class Main {
5       static BookstoreUI ui = new BookstoreUI(); // draws the screens and reads answers
6       static Bookstore store;                    // the shop's products and sales
```

7.6 Method overloading

Method overloading means several methods or constructors with the same name but different parameters; the compiler picks the right one from the arguments. `Book` has two constructors: with an ISBN, and without one (many local books have none, so the user types "-"). `Bookstore` has two `restock` methods: the Restock screen calls `restock(String, int)` with the product ID, which finds the product and calls `restock(Product, int)` to do the work.

Product.java lines 85–93

```
85  // CONSTRUCTOR OVERLOADING 1: a book with an ISBN.
86  public Book(String id, String title, double price, int stock, String author, String isbn) {
…
92  // CONSTRUCTOR OVERLOADING 2: a book without an ISBN. THIS(...) calls the constructor above.
93  public Book(String id, String title, double price, int stock, String author) {
```

Bookstore.java lines 48–58

```
48  // METHOD OVERLOADING: restock by product ID, or by the product object itself.
49  public void restock(String id, int quantity) {
50      restock(get(id), quantity);
51  }
53  // Adds units to the stock and saves.
54  public void restock(Product p, int quantity) {
55      if (quantity < 1) throw new RuntimeException("Quantity must be at least 1.");
56      p.setStock(p.getStock() + quantity);
57      save();
58  }
```

Main.java lines 80–100

```
80  String isbn = ui.askText("ISBN (type - if it has none)");
81  Book book; // SUBCLASS reference
82  if (isbn.equals("-")) book = new Book(id, title, price, stock, author); // overloaded constructor
83  else book = new Book(id, title, price, stock, author, isbn);
…
100 store.restock(p.getId(), ui.askNumber("Quantity to add", 1, 1000000)); // overloaded restock(String,
    int)
```

7.7 Method overriding

Overriding means a subclass writes its own version of a method it received from its superclass or an interface. `Product` only declares `getType()` and `getDetails()`; `Book` and `Stationery` each give their own body. `@Override` asks the compiler to check that the method really exists in the parent, so a spelling mistake becomes a compile error. These

overrides are what make the polymorphism in 7.4 work.

Product.java lines 99–125

```
99  // METHOD OVERRIDING: Book's own versions of Product's abstract methods.
100 @Override
101 public String getType() { return "BOOK"; }
103 @Override
104 public String getDetails() { return "Book by " + author + ", ISBN " + isbn; }
…
121 @Override
122 public String getType() { return "STATIONERY"; }
124 @Override
125 public String getDetails() { return "Stationery by " + brand; }
```

7.8 The this keyword

`this` means "the current object". In `this.id = id` the left side is the object's own field and the right side is the constructor parameter; without `this` the parameter would hide the field. `this(...)` calls another constructor of the same class: the Book constructor without an ISBN passes its values to the full constructor and fills the ISBN with "N/A", so the setup code is written only once. The `Sale` constructor in `Bookstore.java` uses `this` the same way.

Product.java lines 37–95

```
37  // Constructor: stores the values and counts the new product.
38  public Product(String id, String title, double price, int stock) {
39      this.id = id; // THIS: this.id is the object's field, id is the parameter
40      this.title = title;
41      this.price = price;
42      this.stock = stock;
43      createdCount++;
44  }
…
92  // CONSTRUCTOR OVERLOADING 2: a book without an ISBN. THIS(...) calls the constructor above.
93  public Book(String id, String title, double price, int stock, String author) {
94      this(id, title, price, stock, author, "N/A");
95  }
```

7.9 The super keyword

`super` refers to the parent class. A subclass constructor starts with `super(id, title, price, stock)`, which runs the `Product` constructor first: it stores the shared fields and increases the static counter. Only then does the subclass store its own fields (author and ISBN, or brand). The other form, `Trackable.super.getCategory()`, calls a parent interface's method; see 7.11.

Product.java lines 85–116

```
85  // CONSTRUCTOR OVERLOADING 1: a book with an ISBN.
86  public Book(String id, String title, double price, int stock, String author, String isbn) {
87      super(id, title, price, stock); // SUPER: runs the Product constructor first
88      this.author = author;
89      this.isbn = isbn;
90  }
…
112 // Constructor: the Product part is filled first by super(...).
113 public Stationery(String id, String title, double price, int stock, String brand) {
114     super(id, title, price, stock); // SUPER: runs the Product constructor first
115     this.brand = brand;
116 }
```

7.10 Subclass vs superclass reference

A subclass reference (`Book book`) can use everything a Book has, including `getAuthor()`. A superclass reference (`Product product`) can point to any subclass object (upcasting) but only sees the methods declared in `Product`. In Add product the new object is first held as a `Book` or `Stationery` to read its author or brand; then `product = book` stores it as a `Product`. From then on every item is treated the same: `store.add(product)` puts it in the `ArrayList<Product>`,

and `product.getType()` still runs Book's own version.

Main.java lines 65–93

```java
// 3 Add product: asks every detail, then saves a Book or a Stationery.
static void addProduct() {
    char type = ui.askLetter("Type: B = book, S = stationery", "BS");
    String id = ui.askText("ID (short code, e.g. B1)");
    while (store.find(id) != null) {
        ui.error("ID " + id + " is already used. Please choose another.");
        id = ui.askText("ID (short code, e.g. B1)");
    }
    String title = ui.askText("Title");
    double price = ui.askMoney("Price (BDT)", 0);
    int stock = ui.askNumber("Stock", 0, 1000000);
    Product product; // SUPERCLASS reference: can hold a Book or a Stationery
    String extra;
    if (type == 'B') {
        String author = ui.askText("Author");
        String isbn = ui.askText("ISBN (type - if it has none)");
        Book book; // SUBCLASS reference
        if (isbn.equals("-")) book = new Book(id, title, price, stock, author); // overloaded
constructor
        else book = new Book(id, title, price, stock, author, isbn);
        extra = " by " + book.getAuthor(); // getAuthor() works only through the Book reference
        product = book;                    // UPCASTING: the Book is stored as a Product
    } else {
        Stationery item = new Stationery(id, title, price, stock, ui.askText("Brand"));
        extra = " from " + item.getBrand();
        product = item;
    }
    store.add(product);
    showResult(3, "Added " + product.getType() + " \"" + title + "\"" + extra + ".");
}
```

7.11 Diamond problem resolution

Java does not allow `class C extends A, B`, which avoids the diamond problem for classes. With interfaces it can still happen: `Trackable` and `Promotable` both extend `Item` and both give their own `default getCategory()`. `Product` implements both, so Java cannot know which one to use and refuses to compile until `Product` overrides the method. It chooses explicitly with `Trackable.super.getCategory()` and `Promotable.super.getCategory()`; the result, "Tracked stock & Promotable", is shown on the product card (Restock and Sell screenshots).

Product.java lines 4–16

```java
interface Item {
    default String getCategory() { return "Item"; }
}
interface Trackable extends Item {
    @Override
    default String getCategory() { return "Tracked stock"; }
}
interface Promotable extends Item {
    @Override
    default String getCategory() { return "Promotable"; }
}
```

Product.java lines 20–73

```java
public abstract class Product implements Trackable, Promotable, Serializable {
…
// DIAMOND PROBLEM SOLVED: we say exactly which parent versions to use.
@Override
public String getCategory() {
    return Trackable.super.getCategory() + " & " + Promotable.super.getCategory();
}
```

BookstoreUI.java line 163

```java
card("Category: " + p.getCategory());
```

8 Key Code Behind the Screens

Three parts of the code make the program safe to use: saving, checking every answer, and the shop's rules.

8.1 Saving the data

All products and sales live inside one `Bookstore` object. Because the classes implement `Serializable`, `save()` writes the whole object to `bookstore.dat` in one step after every change, and `load()` reads it back when the program starts. If the file is damaged, the program says so and never overwrites it.

Bookstore.java lines 101–118

```
101 // Saves the whole bookstore (products and sales) to the file in one step.
102 private void save() {
103     try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(FILE))) {
104         out.writeObject(this);
105     } catch (IOException e) {
106         throw new RuntimeException("Could not save: " + e.getMessage());
107     }
108 }
110 // Reads the saved bookstore, or starts an empty one the first time.
111 public static Bookstore load() {
112     if (!new File(FILE).exists()) return new Bookstore();
113     try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(FILE))) {
114         return (Bookstore) in.readObject();
115     } catch (Exception e) {
116         throw new RuntimeException("Could not read " + FILE + ": " + e.getMessage());
117     }
118 }
```

8.2 Checking every answer

Every question goes through `askText`: it draws the purple input bar, ignores empty answers and turns `cancel` into an exception that ends the task. `askNumber` (and `askMoney`) keep asking until the answer is a number in the allowed range, so a mistake never cancels the whole task.

BookstoreUI.java lines 81–105

```
81  // Keeps asking until something is typed. Typing "cancel" stops the current task.
82  public String askText(String question) {
83      while (true) {
84          // Draw a purple bar, then \r jumps back so the typing starts right after the question.
85          System.out.print(PURPLE + String.format(" %-78s", question + " >") + "\r " + question + " > ");
86          if (!in.hasNextLine()) System.exit(0); // input was closed
87          String answer = in.nextLine().trim();
88          System.out.print(BODY);
89          if (answer.equalsIgnoreCase("cancel")) throw new RuntimeException("Cancelled. Nothing
    changed.");
90          if (!answer.isEmpty()) return answer;
91      }
92  }
94  // Asks again until the answer is a whole number from min to max.
95  public int askNumber(String question, int min, int max) {
96      while (true) {
97          try {
98              int number = Integer.parseInt(askText(question));
99              if (number >= min && number <= max) return number;
100         } catch (NumberFormatException e) {
101             // not a number, so ask again
102         }
103         error("Please enter a whole number from " + min + " to " + max + ".");
104     }
105 }
```

8.3 Shop rules and errors

`Bookstore` checks its rules again before changing any data: the quantity must be in stock, the cash must cover the total, and members get 5% off through `calculateDiscount(5)`. A broken rule throws a `RuntimeException`; the menu loop in

`Main` catches it, shows it in a red bar and returns to the menu, so the program never crashes.

Bookstore.java lines 66–73

```
66  // Members get 5% off. The total is rounded to 2 decimal places.
67  public double total(Product p, int quantity, boolean member) {
68      if (quantity < 1 || quantity > p.getStock())
69          throw new RuntimeException("Quantity must be from 1 to " + p.getStock() + ".");
70      double total = p.getPrice() * quantity;
71      if (member) total = total - p.calculateDiscount(5) * quantity;
72      return Math.round(total * 100) / 100.0;
73  }
```

Main.java lines 35–37

```
35  } catch (RuntimeException e) {
36      ui.error(e.getMessage()); // show the problem in red, then go back to the menu
37  }
```

9 Limitations and Future Work

Current limitation	Possible improvement
One product per sale	A shopping cart with several products on one receipt
Products cannot be deleted or renamed	Edit and delete screens
Only one copy of the program should use the data file at a time	A database (for example SQLite) that supports several users
The Windows console shows only English letters reliably	A graphical interface (JavaFX) with full Unicode support
No user accounts	Staff login with different permissions
Reports only on screen	Export of reports and receipts to PDF or Excel

10 Conclusion

The Bookstore Inventory Manager meets its objectives. It handles the daily work of a small shop (catalogue, stock, sales, refunds and reports) with clear screens, helpful messages and automatic saving. It is short (635 lines in four files) and uses simple Java, yet all 11 OOP concepts of the course appear inside real features: abstraction and inheritance in the product hierarchy, encapsulation in the protected fields, runtime polymorphism in the product table, overloading in the Book constructors and the restock methods, overriding in `getType`, `getDetails` and `getCategory`, static members in the currency and the session counter, `this` and `super` in the constructors, upcasting when a product is added, and the interface diamond problem in the product category.